

# A Comparative Study of AI Models In Software Development

Chris-Petter Fosslund

28 September 2026

## 1 Introduction

This report serves as a continuation of the previous step which defined frame of what the project aimed to achieve. The contents of this report defines the planned steps and experiments more closely and adds more detail towards the end goal. The first stage of project focuses on three sorting algorithms, bubble sort, merge sort and quick sort. These algorithms provides an early increase in complexity between them, which can prove usefull before moving on to binary search trees, which will increase complexity drastically.

## 2 Hardware environment

The planned hardware to be used for the projects duration has not changed. Due to access limitations more "diverse" hardware will not be used, and for the same reason the configurations used is highly unlikely to change for the duration of the project. All comparative benchmarks and tests will be ran on the same hardware. While it is unlikely that the sorting itself will gain any advantages from more powerful hardware, the same hardware will be used to make it as fair as possible.

| Component | Rig 1                   | Rig 2                        |
|-----------|-------------------------|------------------------------|
| CPU       | Intel Core I5 9600k     | Intel Core I5 10300H         |
| RAM       | 16GB DDR4               | 8GB DDR4                     |
| GPU       | AMD Radeon 6700XT 12 GB | Nvidia Geforce GTX 1650 4 GB |

It is not planned to do any experiments which will rely on the GPU in any way, the GPU specification is included mostly to give a more detailed description of the hardware configurations. As previously mentioned, the first rig is running WSL which with its current setup does not access the dedicated GPU, but rather uses the one integrated in CPU. Due to WSL relying on hardware emulation activated through the CPU, it is unknown wether it would be possible to even allow access to the dedicated GPU within WSL.

### 3 Software environment

For conducting the experiments, a simple testing program is being implemented. Currently this is setup to handle the first sets of experiments concerning sorting algorithms. The framework used is however, easy to expand with more tests for more complex algorithms. The LLMs will be accessed through their specific APIs (No self-hosting or Copilot), which should make the experiment less reliant on specific software and the power of the hardware used.

The experiments will be run on one of two software configurations:

| Rig 1                            | Rig 2           | Software Specs |
|----------------------------------|-----------------|----------------|
| Ubuntu                           | Linux Mint 22.2 | OS             |
| No (WSL with Windows 10 as host) | Yes             | Native         |
| x86-64                           | x86-64          | Architecture   |

In this table, Rig 1 and 2 correspond to the ones listed in the hardware section. Due to the projects nature no other specific software will impact the experiments in any meaningful way.

### 4 Datasets

For the first sets of experiments, two datasets are used. Both are unsorted arrays with numbers ranging from 0-2999. This range of numbers is a completely arbitrary decision, and only serves to create a limited range to make testing an easier task. The first arranges the numbers in a random order, as this is in many cases a realistic scenario a sorting algorithm faces where numbers can appear in an order with not clear pattern. The second is in a reversed order, which should add to the amount of work which the algorithms will have to do, the reason for this is to better measure performance in terms of execution time.

When moving on to different algorithms the datasets used will naturally be expanded. The next step of project will revolve around binary search trees, for this purpose other datasets will be used. The exact contents of these is not yet decided, but the general idea is to use another data-type than integers, such that the key-value association aspect of BSTs can be part of the correctness tests. For this reason, the choice of dataset(s) for this phase require more planning and thought.

### 5 Evaluation and performance metrics

For the experiments, two measurable metrics are used. The first of these is correctness. This is measured by validating that an array has been sorted correctly. By setting an upper and lower limit to which numbers are used, we know what the correctly sorted array is going to look like and therefore, this can be measured by checking if all numbers are in their expected slots and keeping a counter of how many correct elements there are. As previously mentioned, one

of the used datasets is a randomly sorted array, to account for the possibility that this at least in theory could produce an already sorted array, multiple runs will be conducted as to not give any unfair advantage to any LLM model. The second metric will be execution time, this will be more of a fallback metric for mainly two reasons, one : At least in the first set of experiments, it is expected that any difference will be minimal if any. Two: The main idea behind the project is to attempt to determine whether or not AI can be trusted with implementing algorithms independently, therefore, correct implementation matter more than performance.

## 6 Experimental protocols

The way that the experiments are going to work are as follows. Each LLM has been given the task of implementing three sorting algorithms which are bubble sort, merge sort and quick sort. Each implementation will then be given two unsorted arrays, one in a random order and one in a reverse order, both containing 3000 unique numbers. After sorting the result will then be verified to see if the array has been correctly sorted. This is done by incrementing a counter each time a matching element is found between the newly sorted array and a reference array already in the correct order, if that counter reaches 3000, the correctness test is passed. Secondly, should all the LLMs pass the tests, it is also possible to compare execution time. This is measured by a timer which starts just before the function call and ends right after.

## 7 Timeline

Originally the plan was to have the first round of experiments conducted by the assignment 2 deadline, due to time constraints this has not happened. However, the testing framework is very close to completion, meaning by the next milestone, this will be completed. Additionally, the plan is to also conduct a second round of experiments on binary search trees. However, setting an exact date for the second round is difficult considering round 1 is not yet completed.

The missing parts to be able to complete the first round are code generated by GPT and Gemini (Claude is already implemented) and certain tweaks to the testing framework to ensure that it works as intended.

## 8 Discussion

The current experimental design provides a controlled environment in which the ability of LLMs to implement sorting algorithms without direct human intervention can be evaluated. The main idea behind design is to have a testing environment where any differences in results can be attributed to the AI generated code rather than the framework or the hardware. The testing conditions cover two important scenarios any sorting algorithm is likely to encounter, first,

a randomly sorted array and second, one in reverse order, which for certain types of sorting algorithms can be a demanding task. A potential weakness with this method is that it only tests the implementations under certain conditions and therefore cannot serve as a complete assessment for suitability in every plausible scenario.

The use of WSL for one of the hardware configurations could introduce negative differences in performance despite having more powerful hardware. There are two mitigating solutions to this challenge. One, experiments conducted on different hardware will not be compared unless it is directly relevant to an issue/question encountered during experimentation, in this case this will be given consideration when interpreting the results. Two, while not having access to a dedicated GPU in certain cases could be considered a drawback, due to the nature of project and the planned experiments, this is expected to have a negligible impact if any.

The choice of starting with sorting algorithms serves to create a baseline for assessing the capabilities of LLMs to implement algorithms independently. While being an important part of the project as a whole, the true test will be if the findings in the first phase are still applicable when complexity increases both in terms of the algorithms themselves but also the datasets used.